

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

COORDENAÇÃO CENTRAL DE EDUCAÇÃO CONTINUADA

Ciência de Dados e Analytics

MVP – ENGENHARIA DE DADOS

**MVP - PIPELINE DE DADOS PARA ANÁLISE DE INDICADORES ECONÔMICOS E SEUS
IMPACTOS POTENCIAIS SOBRE FINANCIAMENTOS IMOBILIÁRIOS**

Nome da Aluna: Miriam Stal

Matricula: 4052026000329

São Bento do Sul - SC

2026

Sumário

1. Introdução.....	3
2. Contexto de Negócio	3
3. Coleta dos Dados	4
4. Arquitetura e Modelagem	5
5. Implementação do Pipeline.....	7
5.1 Camada Bronze	8
5.2 Camada Silver	20
5.3 Camada Gold.....	46
6. Qualidade dos Dados	55
7. Análises e Resultados	55
8. Conclusão	64
9. Autoavaliação	66
10. Referências.....	68

1. Introdução

Este documento apresenta o desenvolvimento de um MVP construído sobre a Arquitetura Medalhão (Bronze, Silver e Gold), com o objetivo de estruturar um pipeline de dados completo — da ingestão à análise — para o estudo comparativo entre os principais indexadores financeiros do mercado brasileiro: CDI, Selic e IPCA. O relatório descreve as decisões de arquitetura, as etapas de tratamento e qualidade de dados implementadas em cada camada, e as análises finais construídas para simular cenários de financiamento imobiliário indexados a diferentes referenciais. A partir dessa análise, busca-se responder a perguntas de negócio relevantes tanto para instituições financeiras quanto para consumidores: em que períodos a pressão financeira se intensifica e como a volatilidade de cada referencial impacta o custo total do crédito.

2. Contexto de Negócio

Este projeto tem como objetivo analisar o comportamento dos principais indexadores utilizados no mercado financeiro brasileiro e avaliar seus potenciais impactos sobre financiamentos imobiliários. Para isso, foram utilizadas séries históricas do CDI, da Taxa Selic e do IPCA, permitindo comparar sua evolução temporal e construir cenários analíticos voltados ao entendimento do custo potencial do crédito ao longo do tempo.

Perguntas de negócio:

1. Como CDI, Selic e IPCA evoluíram ao longo dos últimos dez anos?
2. Qual teria sido o custo acumulado de um financiamento corrigido pelo CDI em comparação com o IPCA?
3. Qual indexador apresentou maior volatilidade no período analisado?
4. Em quais períodos um financiamento indexado ao CDI apresentaria maior pressão financeira para o consumidor?

3. Coleta dos Dados

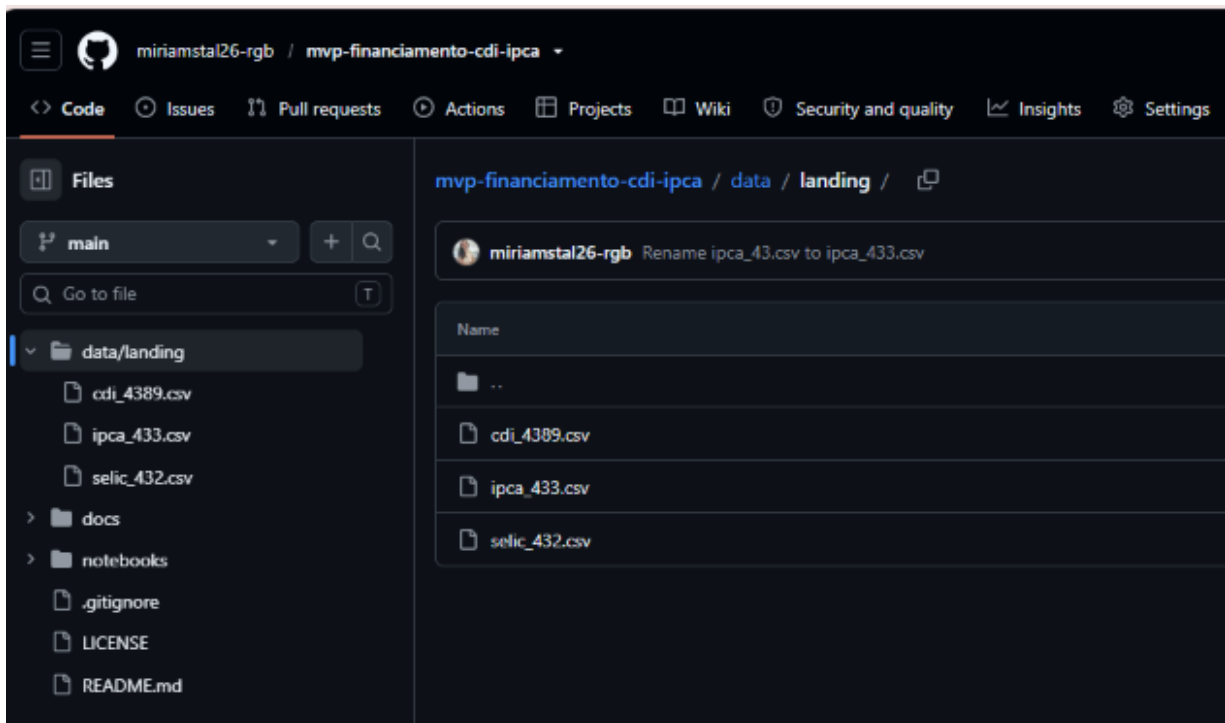
Os dados foram obtidos a partir do [SGS - Sistema Gerenciador de Séries Temporais do Banco Central do Brasil](#). As séries foram exportadas em formato CSV e armazenadas em um repositório GitHub [MVP - Financiamento CDI IPCA](#) utilizado como Landing Zone do projeto. A estratégia de armazenamento dos arquivos em uma Landing Zone baseada em GitHub foi adotada para garantir reprodutibilidade, rastreabilidade e versionamento dos dados utilizados durante o desenvolvimento do MVP.

Séries utilizadas:

- CDI (Código SGS 4389)
- Selic Meta (Código SGS 432)
- IPCA (Código SGS 433)

Estratégia adotada:

- Exportação manual dos arquivos CSV.
- Armazenamento em repositório GitHub.
- Utilização do GitHub como camada Landing para garantir rastreabilidade e versionamento.



SCREENSHOTS DA LANDING ZONE

4. Arquitetura e Modelagem

O projeto utiliza a Arquitetura Medalhão separando os dados em diferentes níveis de refinamento.

A modelagem da solução foi baseada na Arquitetura Medalhão, amplamente utilizada em ambientes Lakehouse. Essa abordagem organiza os dados em múltiplas camadas de refinamento, permitindo separar dados brutos, dados tratados e dados orientados ao consumo analítico.

Fluxo:

SGS → Landing (GitHub) → Bronze → Silver → Gold → Análises

Estrutura planejada:

Landing

- cdi_4389.csv

- selic_432.csv
- ipca_433.csv

Bronze

- bronze_cdi
- bronze_selic
- bronze_ipca

Silver

- dim_calendario
- silver_indicadores_macro
- silver_quality_report

Gold

- gold_evolucao_indicadores
- gold_financiamento_indexado
- gold_risco_indexadores
- gold_simulacao_sac

Justificativa:

A adoção da Arquitetura Medalhão permite separar dados brutos, tratados e analíticos, aumentando a rastreabilidade, governança e organização do pipeline.



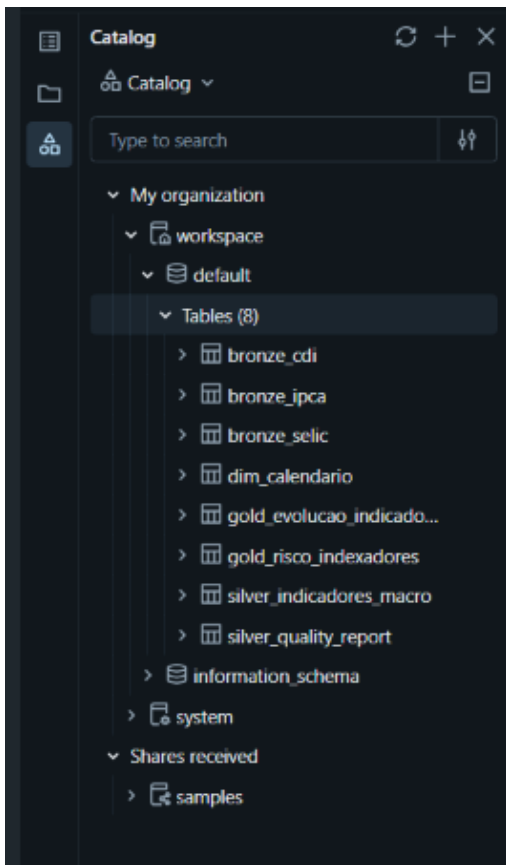
DIAGRAMA DA ARQUITETURA MEDALHÃO

5. Implementação do Pipeline

A implementação foi iniciada no Databricks utilizando notebooks organizados por camada.

Notebooks implementados:

- 01_ingest_bronze
- 02_transform_silver
- 03_build_gold
- 04_analysis



TABELAS PERSISTIDAS NO CATÁLOGO DO DATABRICKS

5.1 Camada Bronze

A Camada Bronze é responsável por armazenar os dados brutos das série histórica do CDI, do IPCA e da Selic, carregados a partir dos arquivos CSV disponibilizados na Landing Zone do projeto, hospedada no GitHub. Nesta etapa, os dados são ingeridos sem transformações significativas, preservando seu formato original para garantir rastreabilidade em relação à fonte.

Abaixo seguem as etapas realizadas:

Importações e leitura dos arquivos CSV diretamente do GitHub.

1. IMPORTAÇÕES

```
▶ ✓ 0402 PM (<1s) 2

# Análise e ingestão
import pandas as pd

# Spark
from pyspark.sql import functions as F
from pyspark.sql.functions import (
    col,
    lit,
    current_timestamp,
    count,
    when
)
```

2. URLS

```
▶ ✓ 0427 PM (<1s) 4

URL_CDI = "https://raw.githubusercontent.com/miriamstal26-rgb/mvp-financiamento-cdi-ipca/refs/heads/main/data/landing/cdi_4389.csv"

URL_SELIC = "https://raw.githubusercontent.com/miriamstal26-rgb/mvp-financiamento-cdi-ipca/refs/heads/main/data/landing/selic_432.csv"

URL_IPCA = "https://raw.githubusercontent.com/miriamstal26-rgb/mvp-financiamento-cdi-ipca/refs/heads/main/data/landing/ipca_433.csv"
```

Ingestão dos dados

3. INGESTÃO DOS DADOS

04:27 PM (<1s) 6

```
# INGESTÃO DO CDI

df_cdi = pd.read_csv(
    URL_CDI,
    sep=";"
)
```

df_cdi: pandas.core.frame.DataFrame = [Data: object, 4389 · Taxa de juros · CDI anualizada base 252 · % a.a.: object]

04:16 PM (<1s) 7

```
# INGESTÃO DA SELIC

df_selic = pd.read_csv(
    URL_SELIC,
    sep=";"
)
```

df_selic: pandas.core.frame.DataFrame = [Data: object, 432 · Taxa de juros · Meta Selic definida pelo Copom · % a.a.: object]

04:16 PM (<1s) 8

```
# INGESTÃO DO IPCA

df_ipca = pd.read_csv(
    URL_IPCA,
    sep=";"
)
```

df_ipca: pandas.core.frame.DataFrame = [Data: object, 433 · Índice nacional de preços ao consumidor-amplo (IPCA) · Var. % mensal: object]

Sequencia de Screenshots da validação estrutural dos dados.

4. VALIDAÇÃO INICIAL

04:18 PM (<1s)

10

```
# CDI
```

```
df_cdi.head()
```

```
df_cdi.tail()
```

```
df_cdi.info()
```

```
df_cdi.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2509 entries, 0 to 2508
```

```
Data columns (total 2 columns):
```

#	Column	Non-Null Count	Dtype
0	Data	2509 non-null	object
1	4389 - Taxa de juros - CDI anualizada base 252 - % a.a.	2509 non-null	object

```
dtypes: object(2)
```

```
memory usage: 39.3+ KB
```

```
Data
```

```
0
```

```
4389 - Taxa de juros - CDI anualizada base 252 - % a.a.
```

```
0
```

```
dtype: int64
```

04:19 PM (<1s)

11

```
# IPCA
```

```
df_ipca.head()
```

```
df_ipca.tail()
```

```
df_ipca.info()
```

```
df_ipca.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 121 entries, 0 to 120
```

```
Data columns (total 2 columns):
```

#	Column	Non-Null Count	Dtype
0	Data	121 non-null	object
1	433 - Índice nacional de preços ao consumidor-amplo (IPCA) - Var. % mensal	121 non-null	object

```
dtypes: object(2)
```

```
memory usage: 2.0+ KB
```

```
Data
```

```
0
```

```
433 - Índice nacional de preços ao consumidor-amplo (IPCA) - Var. % mensal
```

```
0
```

```
dtype: int64
```

```
▶ ✓ 04:20 PM (<1s) 12

# SELIC

df_selic.head()
df_selic.tail()
df_selic.info()
df_selic.isnull().sum()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3653 entries, 0 to 3652
Data columns (total 2 columns):
#   Column                                                                 Non-Null Count  Dtype
---  ---
0   Data                                                                    3653 non-null  object
1   432 - Taxa de juros - Meta Selic definida pelo Copom - % a.a.      3653 non-null  object
dtypes: object(2)
memory usage: 57.2+ KB
Data
432 - Taxa de juros - Meta Selic definida pelo Copom - % a.a.    0
dtype: int64
```

Tratamento de registros não pertencentes às séries temporais.

Durante a inspeção inicial dos dados por meio da função *tail()*, foi identificado que os arquivos exportados pelo Sistema Gerenciador de Séries Temporais (SGS) apresentam um registro adicional ao final da série contendo informações sobre a fonte dos dados.

Exemplos identificados:

- - CDI → Fonte = BCB-Demab
- - Selic → Fonte = Copom
- - IPCA → Fonte = IBGE

▶	✓	04:27 PM (<1s)	14
<code>df_cdi.tail(3)</code>			
Data 4389 - Taxa de juros - CDI anualizada base 252 - % a.a.			
2506	29/06/2026	14,15	
2507	30/06/2026	14,15	
2508	Fonte	BCB-Demab	

▶	✓	04:27 PM (<1s)	15
<code>df_ipca.tail(3)</code>			
Data 433 - Índice nacional de preços ao consumidor-amplio (IPCA) - Var. % mensal			
118	05/2026	0,58	
119	06/2026	0,16	
120	Fonte	IBGE	

▶	✓	04:27 PM (<1s)	16
<code>df_selic.tail(3)</code>			
Data 432 - Taxa de juros - Meta Selic definida pelo Copom - % a.a.			
3650	29/06/2026	14,25	
3651	30/06/2026	14,25	
3652	Fonte	Copom	

Embora essas informações sejam relevantes para fins de documentação e rastreabilidade, elas não representam observações válidas da série temporal analisada. Sua permanência nas tabelas poderia comprometer a consistência estrutural dos dados, especialmente em etapas posteriores de tratamento, conversão de tipos e análises temporais.

Dessa forma, foi implementada uma função reutilizável para remover esses registros de metadados das três bases, preservando apenas as observações efetivamente pertencentes às séries históricas.

Esta transformação foi realizada ainda na camada Bronze por se tratar de uma correção estrutural da fonte de dados, e não de uma transformação analítica ou de negócio.

▶ ✓ 04:29 PM (<1s)

17

```
# FUNÇÃO PARA REMOÇÃO DE REGISTROS DE METADADOS
```

```
def remover_linha_fonte(df):  
    """  
    Remove a linha de metadado 'Fonte'  
    presente ao final dos arquivos exportados pelo SGS.  
    """  
    return df[df.iloc[:, 0] != "Fonte"]
```

▶ ✓ 04:29 PM (<1s)

18

```
# Aplicando tratamento
```

```
df_cdi = remover_linha_fonte(df_cdi)  
df_selic = remover_linha_fonte(df_selic)  
df_ipca = remover_linha_fonte(df_ipca)
```

```
> 3 dataframes: df_cdi, df_ipca, df_selic
```

▶	✓	04:29 PM (<1s)	19
<code>df_cdi.tail(3)</code>			
Data 4389 - Taxa de juros - CDI anualizada base 252 - % a.a.			
2505	26/06/2026	14,15	
2506	29/06/2026	14,15	
2507	30/06/2026	14,15	

▶	✓	04:29 PM (<1s)	20
<code>df_selic.tail(3)</code>			
Data 432 - Taxa de juros - Meta Selic definida pelo Copom - % a.a.			
3649	28/06/2026	14,25	
3650	29/06/2026	14,25	
3651	30/06/2026	14,25	

▶	✓	04:29 PM (<1s)	21
<code>df_ipca.tail(3)</code>			
Data 433 - Índice nacional de preços ao consumidor-amplo (IPCA) - Var. % mensal			
117	04/2026	0,67	
118	05/2026	0,58	
119	06/2026	0,16	

Padronização estrutural das colunas

Após a remoção dos registros de metadados identificados na etapa anterior, foi realizada a padronização dos nomes das colunas das três séries econômicas.

Os arquivos exportados pelo Sistema Gerenciador de Séries Temporais (SGS) apresentam nomes de colunas extensos e específicos para cada indicador, contendo descrições completas das séries. Embora essas informações sejam úteis para consulta humana, elas dificultam a

padronização do pipeline e a reutilização das transformações entre diferentes conjuntos de dados.

Dessa forma, os nomes das colunas foram padronizados para os atributos genéricos *data* e *valor*, criando uma estrutura comum entre as séries de CDI, Selic e IPCA. Essa alteração não modifica o conteúdo dos dados, apenas sua representação estrutural, facilitando as etapas posteriores de transformação e integração dos dados.



```
df_cdi.columns = ["data", "valor"]

df_selic.columns = ["data", "valor"]

df_ipca.columns = ["data", "valor"]
```

Inclusão de metadados de ingestão

Como parte das boas práticas de Engenharia de Dados, foram adicionados metadados de controle e rastreabilidade aos registros da camada Bronze.

Foram incluídos os atributos:

- *data_ingestao*: registra a data e hora em que o dado foi processado pelo pipeline;
- *arquivo_origem*: identifica o arquivo de origem responsável pela geração do registro.

Esses metadados possibilitam auditoria, monitoramento e rastreamento da origem dos dados ao longo das demais camadas da arquitetura. Sua utilização é comum em ambientes Lakehouse e auxilia na governança dos dados, permitindo identificar quando e de onde cada informação foi carregada.

```
04:37 PM (<ts) 25
from datetime import datetime

momento_ingestao = datetime.now()

df_cdi["data_ingestao"] = momento_ingestao
df_cdi["arquivo_origem"] = "cdi_4389.csv"

df_selic["data_ingestao"] = momento_ingestao
df_selic["arquivo_origem"] = "selic_432.csv"

df_ipca["data_ingestao"] = momento_ingestao
df_ipca["arquivo_origem"] = "ipca_433.csv"

04:37 PM (<ts) 26
df_cdi.head()
```

	data	valor	data_ingestao	arquivo_origem
0	01/07/2016	14,13	2026-09-17 19:37:28.756909	cdi_4389.csv
1	04/07/2016	14,13	2026-09-17 19:37:28.756909	cdi_4389.csv
2	05/07/2016	14,13	2026-09-17 19:37:28.756909	cdi_4389.csv
3	06/07/2016	14,13	2026-09-17 19:37:28.756909	cdi_4389.csv
4	07/07/2016	14,13	2026-09-17 19:37:28.756909	cdi_4389.csv

Conversão para DataFrames Spark

Após a etapa de ingestão e tratamento inicial utilizando Pandas, os dados são convertidos para DataFrames Spark para permitir o armazenamento e processamento distribuído dentro do ambiente Databricks.

Essa conversão marca a transição dos dados da etapa de preparação local para a estrutura de processamento utilizada pelo Lakehouse.

```
▶ 04:39 PM (1s) 28
spark_cdi = spark.createDataFrame(df_cdi)

spark_selic = spark.createDataFrame(df_selic)

spark_ipca = spark.createDataFrame(df_ipca)

> 3 dataframes: spark_cdi, spark_ipca, spark_selic

▶ 04:39 PM (<1s) 29
spark_cdi.printSchema()

spark_selic.printSchema()

spark_ipca.printSchema()

root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)

root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)

root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)
```

Persistência das tabelas a camada Bronze

Após a ingestão e preparação inicial dos dados, os DataFrames Spark foram persistidos como tabelas da camada Bronze.

O objetivo dessa etapa é disponibilizar os dados de forma estruturada dentro do ambiente Lakehouse, preservando sua granularidade original e mantendo os metadados de rastreabilidade adicionados durante a ingestão.

As tabelas geradas nesta etapa servirão como fonte para os processos de transformação da camada Silver.

Tabelas persistidas:

- bronze_cdi

- bronze_selic
- bronze_ipca

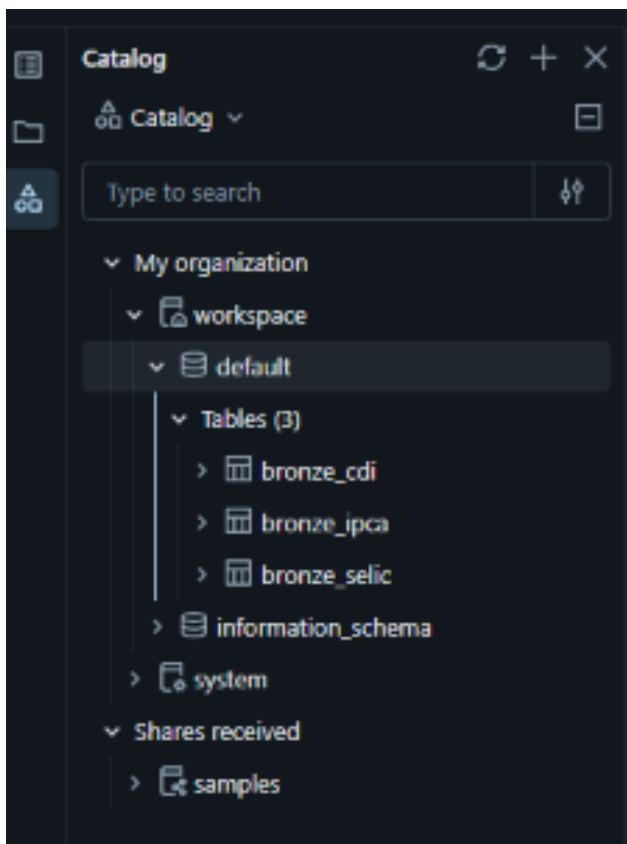
```
▶ 04:41 PM (15s) 32
spark_cdi.write \
  .mode("overwrite") \
  .saveAsTable("bronze_cdi")
> See performance \(1\)
```

```
▶ 04:41 PM (4s) 33
spark_selic.write \
  .mode("overwrite") \
  .saveAsTable("bronze_selic")
> See performance \(1\)
```

```
▶ 04:41 PM (1s) 34
spark_ipca.write \
  .mode("overwrite") \
  .saveAsTable("bronze_ipca")
> See performance \(1\)
```

```
▶ 04:41 PM (1s) 35
spark.sql("SHOW TABLES").show(truncate=false)
> See performance \(2\)
```

database	tableName	isTemporary
default	bronze_cdi	false
default	bronze_ipca	false
default	bronze_selic	false



5.2 Camada Silver

A camada Silver é responsável pela padronização, tratamento e integração dos dados provenientes da camada Bronze, preparando-os para consumo analítico. Nesta etapa, as séries de CDI, Selic e IPCA passam por conversão de tipos (datas para o formato *Date* e indicadores para valores numéricos), padronização do separador decimal e ajuste de suas estruturas temporais.

Como o IPCA é disponibilizado em periodicidade mensal, enquanto CDI e Selic possuem periodicidade diária, foi necessário consolidar todas as séries em uma granularidade mensal comum, adotando o último valor disponível de cada mês como referência. Para viabilizar essa integração, foi criado o atributo `ano_mes`, utilizado como chave temporal entre os indicadores.

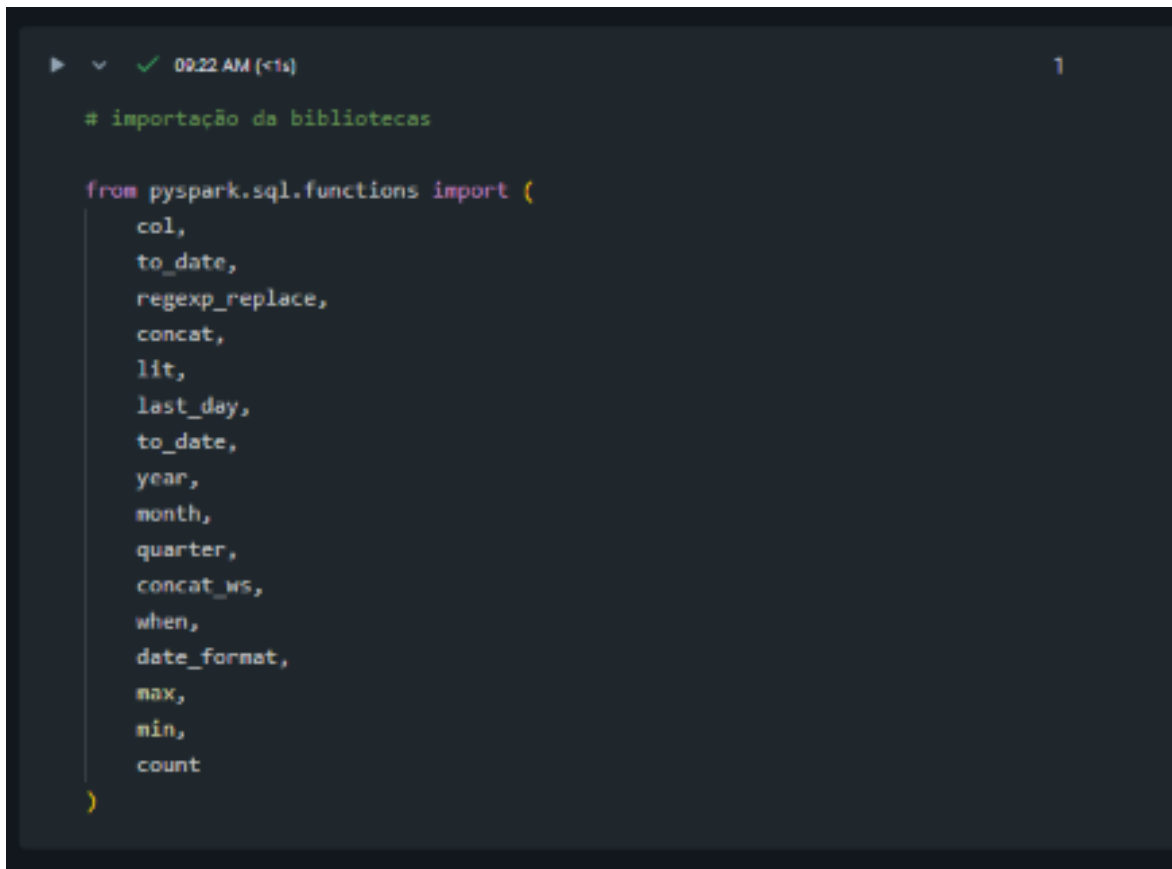
Foi construída também uma dimensão calendário (`dim_calendario`), contendo atributos como ano, mês, trimestre e semestre, seguindo práticas de modelagem dimensional. Ao final

do processo, as três séries são consolidadas na tabela `silver_indicadores_macro`, validada quanto à completude, consistência temporal e presença dos três indicadores em todos os períodos.

As tabelas resultantes desta camada representam a versão tratada e integrada dos dados, servindo de base tanto para as análises de negócio quanto para a construção da camada Gold.

A seguir estão as etapas realizadas e os screenshots da pipeline.

Importação das funções utilizadas

A screenshot of a Jupyter Notebook interface. At the top, there is a status bar showing a play button, a dropdown arrow, a checkmark, the time '09:22 AM', and a refresh icon. The notebook cell contains Python code for importing functions from PySpark's SQL module. The code is as follows:

```
# importação das bibliotecas

from pyspark.sql.functions import (
    col,
    to_date,
    regexp_replace,
    concat,
    lit,
    last_day,
    to_date,
    year,
    month,
    quarter,
    concat_ws,
    when,
    date_format,
    max,
    min,
    count
)
```

Leitura das tabelas persistidas

Nesta etapa são carregadas as tabelas persistidas na camada Bronze, que servirão como fonte para as transformações da camada Silver.

O objetivo desta camada é aplicar regras de qualidade, padronização e integração dos dados, preparando-os para consumo analítico.



```
▶ 08:19 AM (27s) 3  
  
df_cdi = spark.table("bronze_cdi")  
  
df_selic = spark.table("bronze_selic")  
  
df_ipca = spark.table("bronze_ipca")  
  
> 3 dataframes: df_cdi, df_ipca, df_selic
```

Validação da estrutura dos dados

Nesta etapa são avaliadas as estruturas das tabelas provenientes da camada Bronze.

O objetivo é identificar os tipos de dados existentes e planejar as transformações necessárias para a construção da camada Silver.

Nesta camada ocorrerão as conversões de tipos, padronização temporal e integração das séries econômicas.

08:20 AM (15s)

4

```
display(df_cdi)
display(df_selic)
display(df_ipca)
```

> [See performance \(3\)](#)

Table



	Δ_C^B data	Δ_C^B valor	data_ingestao	Δ_C^B arquivo_origem
1	01/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
2	02/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
3	03/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
4	06/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
5	07/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
6	08/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
7	09/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
8	10/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
9	13/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
10	14/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
11	15/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
12	16/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
13	17/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
14	20/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv
15	21/03/2023	13,65	2026-09-17T19:37:28.756+00:...	cdi_4389.csv

2,508 rows 15.391s runtime

Table



	Δ_C^B data	Δ_C^B valor	data_ingestao	Δ_C^B arquivo_origem
1	31/12/2018	6,50	2026-09-17T19:37:28.756+00:...	selic_432.csv
2	01/01/2019	6,50	2026-09-17T19:37:28.756+00:...	selic_432.csv
3	02/01/2019	6,50	2026-09-17T19:37:28.756+00:...	selic_432.csv
4	03/01/2019	6,50	2026-09-17T19:37:28.756+00:...	selic_432.csv

```
▶ 08:21 AM (<1s) 6

df_cdi.printSchema()

df_selic.printSchema()

df_ipca.printSchema()

root
 |-- data: string (nullable = true)
 |-- valor: string (nullable = true)
 |-- data_ingestao: timestamp (nullable = true)
 |-- arquivo_origem: string (nullable = true)

root
 |-- data: string (nullable = true)
 |-- valor: string (nullable = true)
 |-- data_ingestao: timestamp (nullable = true)
 |-- arquivo_origem: string (nullable = true)

root
 |-- data: string (nullable = true)
 |-- valor: string (nullable = true)
 |-- data_ingestao: timestamp (nullable = true)
 |-- arquivo_origem: string (nullable = true)
```

Conversão de Padronização dos Dados

A camada Silver é responsável pela padronização e preparação dos dados para consumo analítico.

Nesta etapa são realizadas transformações que convertem os dados para formatos adequados ao processamento e análise, preservando sua consistência e qualidade.

As transformações aplicadas incluem:

- Conversão de datas do formato texto para o tipo *Date*.
- Conversão dos valores dos indicadores para o tipo numérico.
- Padronização do formato decimal.
- Preparação das séries para integração em uma estrutura analítica única.

▶ ✓ 08:27 AM (1s)

9

```
# dataframe silver do CDI
silver_cdi = (
    df_cdi
    .withColumn(
        "data_referencia",
        to_date(col("data"), "dd/MM/yyyy")
    )
    .withColumn(
        "valor_indicador",
        regexp_replace(col("valor"), ",", ".").cast("double")
    )
)
```

>  silver_cdi: pyspark.sql.connect.dataframe.DataFrame = [data: string, valor: string ... 4 more fields]

▶ ✓ 08:23 AM (<1s)

10

```
silver_cdi.printSchema()
```

```
root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)
|-- data_referencia: date (nullable = true)
|-- valor_indicador: double (nullable = true)
```

08:23 AM (<1s) 10

```
silver_cdi.printSchema()
```

```
root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)
|-- data_referencia: date (nullable = true)
|-- valor_indicador: double (nullable = true)
```

08:24 AM (1s) 11

```
display(
  silver_cdi.select(
    "data",
    "data_referencia",
    "valor",
    "valor_indicador"
  )
)
```

> [See performance \(1\)](#)

Table +

	data	data_referencia	valor	valor_indicador
1	01/03/2023	2023-03-01	13,65	13.65
2	02/03/2023	2023-03-02	13,65	13.65
3	03/03/2023	2023-03-03	13,65	13.65
4	06/03/2023	2023-03-06	13,65	13.65
5	07/03/2023	2023-03-07	13,65	13.65
6	08/03/2023	2023-03-08	13,65	13.65

```
▶ ✓ 08:28 AM (<1s) 14

silver_selic.printSchema()

silver_ipca.printSchema()

root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)
|-- data_referencia: date (nullable = true)
|-- valor_indicador: double (nullable = true)

root
|-- data: string (nullable = true)
|-- valor: string (nullable = true)
|-- data_ingestao: timestamp (nullable = true)
|-- arquivo_origem: string (nullable = true)
|-- data_referencia: date (nullable = true)
|-- valor_indicador: double (nullable = true)
```

Após a validação da conversão realizada para a série CDI, o mesmo procedimento é aplicado às séries Selic e IPCA.

O objetivo desta etapa é garantir que todas as séries econômicas apresentem uma estrutura padronizada e consistente, permitindo sua integração em uma única tabela analítica na camada Silver.

A uniformização dessas estruturas é fundamental para as etapas posteriores de consolidação, análise temporal e modelagem analítica dos dados.

```
▶ 08:30 AM (1s) 16

silver_cdi.select(
  "data_referencia",
  "valor_indicador"
).show(5)

> See performance \(1\)
```

data_referencia	valor_indicador
2016-07-01	14.13
2016-07-04	14.13
2016-07-05	14.13
2016-07-06	14.13
2016-07-07	14.13

only showing top 5 rows

```
▶ 08:31 AM (1s) 17

silver_selic.select(
  "data_referencia",
  "valor_indicador"
).show(5)

> See performance \(1\)
```

data_referencia	valor_indicador
2016-07-01	14.25
2016-07-02	14.25
2016-07-03	14.25
2016-07-04	14.25
2016-07-05	14.25

only showing top 5 rows

Tratamento da Série IPCA

Durante a etapa de transformação foi identificado que a série IPCA apresenta periodicidade mensal, enquanto as séries CDI e Selic são disponibilizadas em periodicidade diária.

Para permitir a integração das séries em uma estrutura analítica comum, foi necessário normalizar a informação temporal do IPCA, assumindo o último dia de cada mês como data de referência.

Exemplo:

07/2016 → 31/07/2016

08/2016 → 31/08/2016

Essa abordagem preserva a granularidade mensal da série e permite sua utilização conjunta com os demais indicadores econômicos.



```
▶ 08:12 AM (1s) 19

silver_ipca.select(
  "data",
  "valor"
).show(10, truncate=False)

> View See performance (1)

+-----+-----+
|data   |valor|
+-----+-----+
|07/2016|0,52 |
|08/2016|0,44 |
|09/2016|0,08 |
|10/2016|0,26 |
|11/2016|0,18 |
|12/2016|0,30 |
|01/2017|0,38 |
|02/2017|0,33 |
|03/2017|0,25 |
|04/2017|0,14 |
+-----+-----+
only showing top 10 rows
```

```
silver_ipca = (  
    df_ipca  
    .withColumn(  
        "data_referencia",  
        last_day(  
            to_date(  
                concat(  
                    lit("01/"),  
                    col("data")  
                ),  
                "dd/MM/yyyy"  
            )  
        )  
    )  
    .withColumn(  
        "valor_indicador",  
        regexp_replace(  
            col("valor"),  
            ",",  
            "."  
        ).cast("double")  
    )  
)
```

> silver_ipca: pyspark.sql.connect.dataframe.DataFrame = [data: string, valor: string ... 4 more fields]

```
08:41 AM (tu) 21

silver_ipca.select(
    "data",
    "data_referencia",
    "valor",
    "valor_indicador"
).show(10, truncate=False)

> See performance \(1\)
```

data	data_referencia	valor	valor_indicador
07/2016	2016-07-31	0,52	0.52
08/2016	2016-08-31	0,44	0.44
09/2016	2016-09-30	0,08	0.08
10/2016	2016-10-31	0,26	0.26
11/2016	2016-11-30	0,18	0.18
12/2016	2016-12-31	0,30	0.3
01/2017	2017-01-31	0,38	0.38
02/2017	2017-02-28	0,33	0.33
03/2017	2017-03-31	0,25	0.25
04/2017	2017-04-30	0,14	0.14

only showing top 10 rows

Com as séries devidamente padronizadas, é criada uma dimensão calendário para enriquecer as análises temporais do projeto.

Embora os indicadores já possuam um atributo de data, a utilização de uma dimensão específica permite disponibilizar informações derivadas como ano, semestre, trimestre e mês de forma padronizada.

```
08:36 AM (<1s) 23

dim_calendario = (
    silver_cdi
    .select("data_referencia")
    .distinct()
    .withColumn("ano", year("data_referencia"))
    .withColumn("mes", month("data_referencia"))
    .withColumn("trimestre", quarter("data_referencia"))
    .withColumn(
        "semestre",
        when(month("data_referencia") <= 6, 1).otherwise(2)
    )
    .withColumn(
        "nome_mes",
        date_format("data_referencia", "MMMM")
    )
    .withColumn(
        "ano_mes",
        concat_ws(
            "-",
            year("data_referencia"),
            month("data_referencia")
        )
    )
)

> dim_calendario: pyspark.sql.connect.dataframe.DataFrame = [data_referencia: date, ano: integer ... 5 more fields]
```

Criação da Dimensão Calendário

- ano;
- mês;
- trimestre;
- semestre;
- ano_mes.

08:36 AM (24) 24

display(dim_calendario)

> See performance (1)

Table +

	data_referencia	ano	mes	trimestre	semestre	nome_mes	ano_mes
1	2023-03-21	2023	3	1	1	March	2023-3
2	2023-05-04	2023	5	2	1	May	2023-5
3	2023-05-05	2023	5	2	1	May	2023-5
4	2023-05-29	2023	5	2	1	May	2023-5
5	2023-06-22	2023	6	2	1	June	2023-6
6	2023-08-01	2023	8	3	2	August	2023-8
7	2023-08-18	2023	8	3	2	August	2023-8
8	2023-10-04	2023	10	4	2	October	2023-10
9	2023-10-09	2023	10	4	2	October	2023-10
10	2023-10-26	2023	10	4	2	October	2023-10
11	2023-11-14	2023	11	4	2	November	2023-11
12	2023-11-17	2023	11	4	2	November	2023-11
13	2023-11-28	2023	11	4	2	November	2023-11
14	2023-12-19	2023	12	4	2	December	2023-12
15	2023-12-20	2023	12	4	2	December	2023-12

2,508 rows 1.906s runtime

Consolidação Temporal dos Indicadores

As séries CDI e Selic são disponibilizadas em periodicidade diária, enquanto o IPCA é disponibilizado em periodicidade mensal.

Para permitir a comparação entre os indicadores em uma mesma granularidade temporal, foi adotado o último valor disponível de cada mês para as séries CDI e Selic.

Essa abordagem é amplamente utilizada no mercado financeiro e em análises econômicas, pois o fechamento do período representa o valor vigente ao final de cada mês, servindo como referência para comparações temporais, elaboração de relatórios e acompanhamento da evolução dos indicadores.

Para a série IPCA, foi utilizada a última data de cada mês como data de referência, garantindo consistência temporal entre todos os indicadores analisados.

Após essa consolidação, todas as séries passam a compartilhar a mesma granularidade mensal, possibilitando sua integração em uma única estrutura analítica para as etapas posteriores do projeto.

▶ ✓ 08:44 AM (<1s) 26

```
cdi_mensal_base = (  
    silver_cdi  
    .withColumn("ano", year("data_referencia"))  
    .withColumn("mes", month("data_referencia"))  
)
```

> cdi_mensal_base: pyspark.sql.connect.dataframe.DataFrame = [data: string, valor: string ... 6 more fields]

▶ ✓ 08:45 AM (<1s) 27

```
selic_mensal_base = (  
    silver_selic  
    .withColumn("ano", year("data_referencia"))  
    .withColumn("mes", month("data_referencia"))  
)
```

> selic_mensal_base: pyspark.sql.connect.dataframe.DataFrame = [data: string, valor: string ... 6 more fields]

▶ ✓ 08:45 AM (<1s) 28

```
ultima_data_cdi = (  
    cdi_mensal_base  
    .groupBy("ano", "mes")  
    .agg(  
        max("data_referencia").alias("data_referencia")  
    )  
)
```

> ultima_data_cdi: pyspark.sql.connect.dataframe.DataFrame = [ano: integer, mes: integer ... 1 more field]

▶ ✓ 08:45 AM (<1s) 29

```
ultima_data_selic = (  
    selic_mensal_base  
    .groupBy("ano", "mes")  
    .agg(  
        max("data_referencia").alias("data_referencia")  
    )  
)
```

> ultima_data_selic: pyspark.sql.connect.dataframe.DataFrame = [ano: integer, mes: integer ... 1 more field]

08:45 AM (<1s)

30

```
cdi_mensal = (  
    ultima_data_cdi  
    .join(  
        cdi_mensal_base.select(  
            "data_referencia",  
            "valor_indicador"  
        ),  
        "data_referencia",  
        "inner"  
    )  
)
```

> cdi_mensal: pyspark.sql.connect.dataframe.DataFrame = [data_referencia: date, ano: integer ... 2 more fields]

08:46 AM (<1s)

31

```
selic_mensal = (  
    ultima_data_selic  
    .join(  
        selic_mensal_base.select(  
            "data_referencia",  
            "valor_indicador"  
        ),  
        "data_referencia",  
        "inner"  
    )  
)
```

> selic_mensal: pyspark.sql.connect.dataframe.DataFrame = [data_referencia: date, ano: integer ... 2 more fields]

08:48 AM (2s)

32

```
display(cdi_mensal)
```

> [See performance \(1\)](#)

Table



	data_referencia	ano	mes	1.2 valor_indicador
1	2025-08-29	2025	8	14.9
2	2026-04-30	2026	4	14.4
3	2023-10-31	2023	10	12.65
4	2025-09-30	2025	9	14.9
5	2023-06-30	2023	6	13.65
6	2024-01-31	2024	1	11.65
7	2025-03-31	2025	3	14.15
8	2025-04-30	2025	4	14.15
9	2025-07-31	2025	7	14.9
10	2026-03-31	2026	3	14.65
11	2024-06-28	2024	6	10.4
12	2026-06-30	2026	6	14.15
13	2024-05-31	2024	5	10.4
14	2023-09-29	2023	9	12.65
15	2025-12-31	2025	12	14.9



120 rows

1.999s runtime

08:46 AM (1s)

33

```
display(selic_mensal)
```

> [See performance \(1\)](#)

Table



	data_referencia	ano	mes	1.2 valor_indicador
1	2019-10-31	2019	10	5
2	2020-06-30	2020	6	2.25
3	2021-01-31	2021	1	2
4	2019-04-30	2019	4	6.5
5	2020-01-31	2020	1	4.5
6	2020-04-30	2020	4	3.75
7	2019-01-31	2019	1	6.5
8	2020-09-30	2020	9	2
9	2019-02-28	2019	2	6.5
10	2020-02-29	2020	2	4.25
11	2020-03-31	2020	3	3.75
12	2020-10-31	2020	10	2
13	2021-06-30	2021	6	4.25
14	2018-12-31	2018	12	6.5
15	2021-04-30	2021	4	2.75



120 rows

1.248s runtime

08:45 AM (ts) 33

```
display(selic_mensal)
```

> [See performance \(1\)](#)

Table +

	📅 data_referencia	1º ano	1º mes	1.2 valor_indicador
1	2019-10-31	2019	10	5
2	2020-06-30	2020	6	2.25
3	2021-01-31	2021	1	2
4	2019-04-30	2019	4	6.5
5	2020-01-31	2020	1	4.5
6	2020-04-30	2020	4	3.75
7	2019-01-31	2019	1	6.5
8	2020-09-30	2020	9	2
9	2019-02-28	2019	2	6.5
10	2020-02-29	2020	2	4.25
11	2020-03-31	2020	3	3.75
12	2020-10-31	2020	10	2
13	2021-06-30	2021	6	4.25
14	2018-12-31	2018	12	6.5
15	2021-04-30	2021	4	2.75

120 rows 1.248s runtime

08:52 AM (3s) 41

```
print("CDI:", cdi_mensal.count())
print("Selic:", selic_mensal.count())
print("IPCA:", silver_ipca.count())
```

> [See performance \(3\)](#)

```
CDI: 120
Selic: 120
IPCA: 120
```

Validação dos dados:

- o primeiro mês está correto;

- o último mês está correto;
- não há meses duplicados;
- CDI e Selic realmente ficaram mensais.

Foi adotado o último valor disponível de cada mês para as séries CDI e Selic.

Essa abordagem é amplamente utilizada no mercado financeiro e em análises econômicas, pois representa o último valor efetivamente observado antes do encerramento de cada período de referência.

Como alguns indicadores financeiros não possuem registros em fins de semana e feriados, a data do último registro disponível pode não coincidir com o último dia do calendário. Nesses casos, considera-se a última observação efetivamente registrada.

Para viabilizar a integração dos indicadores em uma mesma granularidade temporal, será criado o atributo `ano_mes`, que representará a competência mensal de cada observação. Essa abordagem permitirá a consolidação das séries CDI, Selic e IPCA por período de referência, independentemente de pequenas diferenças nas datas de fechamento dos registros.

```
▶ 08:57 AM (<1s) 43
cdi_mensal = (
    cdi_mensal
    .withColumn(
        "ano_mes",
        date_format("data_referencia", "yyyy-MM")
    )
)

> cdi_mensal: pyspark.sql.connect.dataframe.DataFrame = [data_referencia: date, ano: integer ... 3 more fields]

▶ 08:57 AM (<1s) 44
selic_mensal = (
    selic_mensal
    .withColumn(
        "ano_mes",
        date_format("data_referencia", "yyyy-MM")
    )
)

> selic_mensal: pyspark.sql.connect.dataframe.DataFrame = [data_referencia: date, ano: integer ... 3 more fields]

▶ 08:57 AM (<1s) 45
silver_ipca = (
    silver_ipca
    .withColumn(
        "ano_mes",
        date_format("data_referencia", "yyyy-MM")
    )
)

> silver_ipca: pyspark.sql.connect.dataframe.DataFrame = [data: string, valor: string ... 5 more fields]
```

Integração das Séries

- criação da silver_indicadores_macro;
- integração por ano_mes.

Após a padronização e consolidação temporal dos indicadores, as séries CDI, Selic e IPCA são integradas em uma única estrutura analítica.

A integração é realizada utilizando o atributo ano_mes, que representa a competência temporal de cada observação.

Essa abordagem permite comparar os indicadores em uma mesma granularidade temporal, independentemente de diferenças na periodicidade de coleta ou na data exata de fechamento das séries.

A tabela resultante servirá como principal fonte de informação para as análises e para a construção da camada Gold.



```
▶ 09:18 AM (<1s) 54

# Tabela consolidada

silver_indicadores_macro = (
    cdi_final
    .join(
        selic_final,
        "ano_mes",
        "inner"
    )
    .join(
        ipca_final,
        "ano_mes",
        "inner"
    )
)

> silver_indicadores_macro: pyspark.sql.connect.dataframe.DataFrame = [ano_mes: string, cdi_percentual: double ... 2 more fields]
```

09:19 AM (3s) 55

```
display(
  silver_indicadores_macro.orderBy("ano_mes")
)
```

> [See performance \(1\)](#)

Table +

	ano_mes	1.2 cdi_percentual	1.2 selic_percentual	1.2 ipca_percentual
1	2016-07	14.13	14.25	0.52
2	2016-08	14.13	14.25	0.44
3	2016-09	14.13	14.25	0.08
4	2016-10	13.88	14	0.26
5	2016-11	13.88	14	0.18
6	2016-12	13.63	13.75	0.3
7	2017-01	12.88	13	0.38
8	2017-02	12.13	12.25	0.33
9	2017-03	12.13	12.25	0.25
10	2017-04	11.13	11.25	0.14
11	2017-05	11.13	11.25	0.31
12	2017-06	10.14	10.25	-0.23
13	2017-07	9.14	9.25	0.24
14	2017-08	9.14	9.25	0.19
15	2017-09	8.14	8.25	0.16

120 rows 2.792s runtime

Após a integração dos indicadores econômicos, foram realizadas validações para verificar a consistência temporal dos dados e confirmar a manutenção do período completo de análise.

As verificações contemplaram:

- Quantidade de registros consolidados;
- Período inicial e final da série;
- Presença dos três indicadores em todos os meses;
- Consistência da chave temporal ano_mes.

Os resultados confirmaram a integração bem-sucedida das séries CDI, Selic e IPCA em uma única estrutura analítica mensal.

Persistencia da Camada Silver

Após a conclusão das etapas de transformação, padronização e integração dos indicadores econômicos, as tabelas da camada Silver são persistidas no ambiente Lakehouse.

Essa etapa disponibiliza conjuntos de dados prontos para consumo analítico e para a construção da camada Gold, preservando a rastreabilidade das transformações realizadas ao longo do pipeline.

As tabelas geradas nesta camada representam a versão tratada e integrada dos dados, servindo como base para análises de negócio e simulações financeiras.

```
▶ ✓ 09:27 AM (9s) 67
silver_quality_report.write \
  .mode("overwrite") \
  .saveAsTable("silver_quality_report")
> View See performance (1)

▶ ✓ 09:27 AM (4s) 68
dim_calendario.write \
  .mode("overwrite") \
  .saveAsTable("dim_calendario")
> View See performance (1)

▶ ✓ 09:27 AM (5s) 69
silver_indicadores_macro.write \
  .mode("overwrite") \
  .saveAsTable("silver_indicadores_macro")
> View See performance (1)

▶ ✓ 09:28 AM (1s) 70
spark.sql("SHOW TABLES").show(truncate=False)
> View See performance (2)

+-----+-----+-----+
|database|tableName      |isTemporary|
+-----+-----+-----+
|default |bronze_cdi      |false      |
|default |bronze_ipca     |false      |
|default |bronze_selic    |false      |
|default |dim_calendario  |false      |
|default |silver_indicadores_macro|false      |
|default |silver_quality_report |false      |
+-----+-----+-----+
```

Considerações Sobre a Consolidação Temporal dos Indicadores

Durante a construção da camada Silver, foi realizada uma análise das características temporais de cada indicador econômico utilizado no projeto.

IPCA: A série do IPCA já é disponibilizada em periodicidade mensal pelo Banco Central. Dessa forma, não foi necessária nenhuma agregação adicional dos valores. Apenas foi realizado o tratamento da data de referência para padronização da estrutura temporal da série.

Selic: A série da Selic é disponibilizada em periodicidade diária. Para permitir sua integração com os demais indicadores em uma mesma granularidade temporal, foi adotado o último valor disponível de cada mês.

Essa abordagem é amplamente utilizada no mercado financeiro e em análises econômicas, pois representa o valor vigente ao encerramento de cada período de referência.

CDI: A série do CDI também é disponibilizada em periodicidade diária. Assim como realizado para a Selic, foi utilizado o último valor disponível de cada mês para compor a estrutura analítica mensal da camada Silver.

Entretanto, é importante destacar que o CDI possui aplicação direta em operações financeiras indexadas e pode demandar cálculos de acumulação para representar corretamente seu impacto ao longo do tempo.

Por esse motivo, eventuais cálculos de taxa mensal equivalente e acumulações financeiras serão realizados na camada Gold, etapa responsável pela aplicação das regras de negócio e simulações de financiamento.

Integração dos Indicadores

Como indicadores financeiros podem não possuir registros em fins de semana e feriados, as datas de fechamento dos registros podem não coincidir exatamente com o último dia do calendário.

Para viabilizar a consolidação dos dados, foi criado o atributo `ano_mes`, representando a competência mensal de cada observação. A integração entre CDI, Selic e IPCA foi realizada utilizando essa chave temporal, garantindo comparabilidade entre os indicadores independentemente de pequenas diferenças nas datas de referência.

5.3 Camada Gold

A camada Gold é responsável pela aplicação das regras de negócio e pela disponibilização dos dados em formatos adequados para análise.

Nesta etapa são carregadas as tabelas produzidas na camada Silver, que servirão como base para a construção dos indicadores e simulações do projeto.

As tabelas geradas nesta camada serão utilizadas posteriormente na análise exploratória e na resposta às perguntas de negócio definidas para o MVP.

A seguir estão descritas as etapas e screenshot do notebook

Importação das funções PySpark

A screenshot of a Jupyter Notebook interface. At the top, there is a status bar showing a green checkmark, the text "24 hours ago (<1s)", and a page number "2". Below this, the code cell contains the following Python code:

```
# importações

from pyspark.sql.functions import (
    avg,
    min,
    max,
    stddev,
    lit,
    round,
    col,
    to_date,
    regexp_replace
)
```

Leitura das tabelas da Camada Silver

A camada Gold é responsável pela aplicação das regras de negócio e pela disponibilização dos dados em formatos adequados para análise.

Nesta etapa são carregadas as tabelas produzidas na camada Silver, que servirão como base para a construção dos indicadores e simulações do projeto.

As tabelas geradas nesta camada serão utilizadas posteriormente na análise exploratória e na resposta às perguntas de negócio definidas para o MVP.

▶

✓ Yesterday (26s)

3

```
silver_indicadores_macro = spark.table(  
    "silver_indicadores_macro"  
)  
  
silver_quality_report = spark.table(  
    "silver_quality_report"  
)  
  
dim_calendario = spark.table(  
    "dim_calendario"  
)  

```

> 3 dataframes: dim_calendario, silver_indicadores_macro, silver_quality_report

▶

✓ Yesterday (16s)

4

```
display(silver_indicadores_macro)
```

> [See performance \(1\)](#)

	ano_mes	1.2 cdi_percentual	1.2 selic_percentual	1.2 ipca_percentual
1	2023-12	11.65	11.75	0.56
2	2024-01	11.65	11.75	0.42
3	2024-02	11.15	11.25	0.83
4	2024-03	10.65	10.75	0.16
5	2024-04	10.65	10.75	0.38
6	2024-05	10.4	10.5	0.46
7	2024-06	10.4	10.5	0.21
8	2024-07	10.4	10.5	0.38
9	2024-08	10.4	10.5	-0.02
10	2024-09	10.65	10.75	0.44
11	2024-10	10.65	10.75	0.56
12	2024-11	11.15	11.25	0.39
13	2024-12	12.15	12.25	0.52
14	2025-01	13.15	13.25	0.16
15	2025-02	13.15	13.25	1.31

↓

120 rows

15.899s runtime

Yesterday (2s) 5

```
display(silver_quality_report)
```

See performance (1)

Table +

	$\mathbb{R}^2$ quantidade_registros	$\mathbb{C}$ data_inicio	$\mathbb{C}$ data_fim	$\mathbb{A}^0$ indicador
1	120	2016-07-31	2026-06-30	Selic
2	120	2016-07-31	2026-06-30	Ipca
3	120	2016-07-29	2026-06-30	CDI

3 rows 2.441s runtime

Yesterday (2s) 6

```
display(din_calendario)
```

See performance (1)

Table +

	$\mathbb{C}$ data_referencia	$\mathbb{R}^2$ ano	$\mathbb{R}^2$ mes	$\mathbb{R}^2$ trimestre	$\mathbb{R}^2$ semestre	$\mathbb{A}^0$ nome_mes	$\mathbb{A}^0$ ano_mes
1	2023-03-21	2023	3	1	1	March	2023-3
2	2023-05-04	2023	5	2	1	May	2023-5
3	2023-05-05	2023	5	2	1	May	2023-5
4	2023-05-29	2023	5	2	1	May	2023-5
5	2023-06-22	2023	6	2	1	June	2023-6
6	2023-08-01	2023	8	3	2	August	2023-8
7	2023-08-18	2023	8	3	2	August	2023-8
8	2023-10-04	2023	10	4	2	October	2023-10
9	2023-10-09	2023	10	4	2	October	2023-10
10	2023-10-26	2023	10	4	2	October	2023-10
11	2023-11-14	2023	11	4	2	November	2023-11
12	2023-11-17	2023	11	4	2	November	2023-11
13	2023-11-28	2023	11	4	2	November	2023-11
14	2023-12-19	2023	12	4	2	December	2023-12
15	2023-12-20	2023	12	4	2	December	2023-12

2,508 rows 1.826s runtime

Construção da Tabela *gold_evolucao_indicadores*

Esta tabela tem como objetivo disponibilizar uma visão consolidada da evolução dos indicadores econômicos utilizados no estudo.

A estrutura resultante servirá como base para análises temporais, comparações entre os indicadores e construção de visualizações analíticas na etapa de exploração dos dados.

```
▶ ✓ Yesterday (<1s) 8

gold_evolucao_indicadores = (
    silver_indicadores_macro
    .orderBy("ano_mes")
)

> gold_evolucao_indicadores: pyspark.sql.connect.dataframe.DataFrame = [ano_mes: string, cdi_percentual: double ... 2 more fields]
```

```
▶ ✓ Yesterday (2s) 9

display(gold_evolucao_indicadores)
> See performance \(1\)
```

	ano_mes	1.2 cdi_percentual	1.2 selic_percentual	1.2 ipca_percentual
1	2016-07	14.13	14.25	0.52
2	2016-08	14.13	14.25	0.44
3	2016-09	14.13	14.25	0.08
4	2016-10	13.88	14	0.26
5	2016-11	13.88	14	0.18
6	2016-12	13.63	13.75	0.3
7	2017-01	12.88	13	0.38
8	2017-02	12.13	12.25	0.33
9	2017-03	12.13	12.25	0.25
10	2017-04	11.13	11.25	0.14
11	2017-05	11.13	11.25	0.31
12	2017-06	10.14	10.25	-0.23
13	2017-07	9.14	9.25	0.24
14	2017-08	9.14	9.25	0.19
15	2017-09	8.14	8.25	0.16

↓ 120 rows 2.206s runtime

Persistencia da tabela *gold_evolucao_indicadores*



```
gold_evolucao_indicadores.write \
    .mode("overwrite") \
    .saveAsTable("gold_evolucao_indicadores")
```

> [See performance \(1\)](#)

Construção da Tabela *gold_risco_indexadores*

Esta tabela tem como objetivo consolidar métricas estatísticas dos indicadores econômicos analisados no projeto.

Os indicadores calculados permitem avaliar o comportamento histórico de cada série e apoiar a análise de volatilidade dos indexadores.

As métricas consideradas são:

- Média;
- Valor Mínimo;
- Valor Máximo;
- Desvio Padrão.

Esses indicadores serão utilizados posteriormente para identificar qual série apresentou maior variabilidade ao longo do período analisado.

▶ ✓ Yesterday (1s)

12

```
risco_cdi = (  
    silver_indicadores_macro  
    .agg(  
        avg("cdi_percentual").alias("media"),  
        min("cdi_percentual").alias("minimo"),  
        max("cdi_percentual").alias("maximo"),  
        stddev("cdi_percentual").alias("desvio_padrao")  
    )  
    .withColumn("indicador", lit("CDI"))  
)
```

▶  risco_cdi: pyspark.sql.connect.dataframe.DataFrame = [media: double, minimo: double ... 3 more fields]

▶ ✓ Yesterday (< 1s)

13

```
risco_selic = (  
    silver_indicadores_macro  
    .agg(  
        avg("selic_percentual").alias("media"),  
        min("selic_percentual").alias("minimo"),  
        max("selic_percentual").alias("maximo"),  
        stddev("selic_percentual").alias("desvio_padrao")  
    )  
    .withColumn("indicador", lit("Selic"))  
)
```

▶  risco_selic: pyspark.sql.connect.dataframe.DataFrame = [media: double, minimo: double ... 3 more fields]

▶ ✓ Yesterday (< 1s)

14

```
risco_ipca = (  
    silver_indicadores_macro  
    .agg(  
        avg("ipca_percentual").alias("media"),  
        min("ipca_percentual").alias("minimo"),  
        max("ipca_percentual").alias("maximo"),  
        stddev("ipca_percentual").alias("desvio_padrao")  
    )  
    .withColumn("indicador", lit("IPCA"))  
)
```

▶  risco_ipca: pyspark.sql.connect.dataframe.DataFrame = [media: double, minimo: double ... 3 more fields]

▶ ✓ Yesterday (<1s)

15

```
gold_risco_indexadores = (  
    risco_cdi  
    .unionByName(risco_selic)  
    .unionByName(risco_ipca)  
)
```

>  gold_risco_indexadores: pyspark.sql.connect.dataframe.DataFrame = [media: double, minimo: double ... 3 more fields]

▶ ✓ Yesterday (<1s)

18

```
gold_risco_indexadores = (  
    gold_risco_indexadores  
    .withColumn("media", round("media", 4))  
    .withColumn("minimo", round("minimo", 4))  
    .withColumn("maximo", round("maximo", 4))  
    .withColumn("desvio_padrao", round("desvio_padrao", 4))  
)
```

>  gold_risco_indexadores: pyspark.sql.connect.dataframe.DataFrame = [media: double, minimo: double ... 3 more fields]

▶ ✓ Yesterday (5s)

19

```
gold_risco_indexadores.write \  
    .mode("overwrite") \  
    .saveAsTable("gold_risco_indexadores")
```

> [View performance \(1\)](#)

Yesterday (1s) 20

```
display(gold_risco_indexadores)
```

[See performance \(1\)](#)

	1.2 media	1.2 minimo	1.2 maximo	1.2 desvio_padrao	A _B _c indicador
1	9.4802	1.9	14.9	4.2299	CDI
2	9.5833	2	15	4.2308	Selic
3	0.4093	-0.68	1.62	0.382	IPCA

3 rows 1.368s runtime

Limitações identificados na modelagem de financiamento

Na arquitetura desenhada foi prevista a criação de mais duas tabelas, *gold_financiamento_indexado* e *gold_simulacao_sac* visando responder a pergunta de negócio “2. Qual teria sido o custo acumulado de um financiamento corrigido pelo CDI em comparação com o IPCA?”

Porém no decorrer da construção identificou-se uma limitação na série SGS 4389 - CDI, que foi selecionada por sua aderência aos objetivos de análise temporal, comparação entre indicadores econômicos e avaliação de risco, atendendo adequadamente às necessidades analíticas propostas para este estudo. No entanto, por se tratar de uma taxa anualizada base 252 dias úteis, sua utilização direta na modelagem de uma taxa mensal efetiva para simulação de financiamentos demandaria tratamentos adicionais e séries complementares específicas para esse fim.

Considerando a necessidade de preservar a consistência metodológica dos resultados, optou-se por não realizar uma simulação financeira baseada em premissas que poderiam comprometer a interpretação dos dados.

Conclusão da camada Gold

```
▶ ✓ 24 hours ago (1s) 22
spark.sql("SHOW TABLES").show(truncate=false)
> See performance \(2\)
```

database	tableName	isTemporary
default	bronze_cdi	false
default	bronze_ipca	false
default	bronze_selic	false
default	din_calendario	false
default	gold_evolucao_indicadores	false
default	gold_risco_indexadores	false
default	silver_indicadores_macro	false
default	silver_quality_report	false

A camada Gold foi responsável por transformar os dados consolidados da camada Silver em estruturas orientadas ao negócio.

Como resultado, foram construídas as seguintes tabelas analíticas:

gold_evolucao_indicadores, utilizada para análise da evolução histórica dos indicadores econômicos;

gold_risco_indexadores, utilizada para avaliação do comportamento estatístico e da volatilidade dos indicadores analisados.

Durante o desenvolvimento desta camada também foi realizada uma avaliação da possibilidade de simulação de financiamentos indexados ao CDI e ao IPCA. Entretanto, devido às características metodológicas da série CDI utilizada (SGS 4389), optou-se por não construir essa simulação, preservando a consistência analítica dos resultados apresentados.

As tabelas produzidas nesta etapa servirão como fonte para a construção das análises, visualizações e conclusões do projeto.

6. Qualidade dos Dados

A qualidade dos dados foi avaliada desde a etapa de ingestão. Foram executadas verificações estruturais, análises de completude dos registros e identificação de inconsistências presentes nos arquivos originais.

Durante a análise na camada Bronze foram executadas verificações utilizando *head()*, *tail()*, *info()* e validações de estrutura.

Já na camada Silver Após a consolidação das séries econômicas, foi elaborado um relatório resumido de qualidade para validar a cobertura temporal e a quantidade de registros disponíveis para cada indicador.

O objetivo desse relatório é verificar a consistência dos dados utilizados no projeto, permitindo identificar possíveis perdas de registros durante as etapas de transformação da camada Silver.

As validações realizadas incluem:

Quantidade total de registros;

Data inicial da série;

Data final da série;

Cobertura temporal disponível para análise.

7. Análises e Resultados

Nesta etapa são analisados os dados preparados ao longo do pipeline desenvolvido neste projeto.

As análises têm como objetivo responder às perguntas de negócio definidas na fase de planejamento, utilizando as tabelas analíticas produzidas na camada Gold.

Perguntas avaliadas:

1. Como CDI, Selic e IPCA evoluíram ao longo dos últimos dez anos?

2. Qual teria sido o custo acumulado de um financiamento corrigido pelo CDI em comparação com o IPCA?
3. Qual indexador apresentou maior volatilidade no período analisado?
4. Em quais períodos um financiamento indexado ao CDI apresentaria maior pressão financeira para o consumidor?

Leitura das tabelas da Gold

```
▶ 23 hours ago (2s) 3

gold_evolucao_indicadores = spark.table(
  "gold_evolucao_indicadores"
)

gold_risco_indexadores = spark.table(
  "gold_risco_indexadores"
)

> 2 dataframes: gold_evolucao_indicadores, gold_risco_indexadores
```

Evolução dos Indicadores Econômicos

A primeira análise busca compreender a evolução histórica dos indicadores CDI, Selic e IPCA ao longo do período estudado.

Para isso, será utilizada a tabela *gold_evolucao_indicadores*, que consolida os indicadores em granularidade mensal.

```
▶ 24 hours ago (<1s)

gold_evolucao_indicadores.printSchema()

root
|-- ano_mes: string (nullable = true)
|-- cdi_percentual: double (nullable = true)
|-- selic_percentual: double (nullable = true)
|-- ipca_percentual: double (nullable = true)
```

24 hours ago (fig) 6

```
display(gold_evolucao_indicadores)
```

> [See performance \(1\)](#)

Table +

	ano_mes	1.2 cdi_percentual	1.2 selic_percentual	1.2 ipca_percentual
1	2016-07	14.13	14.25	0.52
2	2016-08	14.13	14.25	0.44
3	2016-09	14.13	14.25	0.08
4	2016-10	13.88	14	0.26
5	2016-11	13.88	14	0.18
6	2016-12	13.63	13.75	0.3
7	2017-01	12.88	13	0.38
8	2017-02	12.13	12.25	0.33
9	2017-03	12.13	12.25	0.25
10	2017-04	11.13	11.25	0.14
11	2017-05	11.13	11.25	0.31
12	2017-06	10.14	10.25	-0.23
13	2017-07	9.14	9.25	0.24
14	2017-08	9.14	9.25	0.19
15	2017-09	8.14	8.25	0.16

120 rows 6.053s runtime

1. Como CDI, Selic e IPCA evoluíram ao longo dos últimos dez anos?

Esta análise tem como objetivo comparar a evolução temporal dos três principais indicadores utilizados neste estudo.

A observação simultânea das séries históricas permite identificar tendências, mudanças de ciclo econômico e períodos de maior ou menor estabilidade econômica.



Observação Metodológica

Os indicadores analisados possuem naturezas e periodicidades distintas.

CDI e Selic são representados por taxas anualizadas (% a.a.), enquanto o IPCA representa a variação percentual dos preços ao consumidor em base mensal (% a.m.).

Dessa forma, o objetivo desta análise não é comparar diretamente a magnitude absoluta dos valores observados, mas sim identificar tendências, ciclos econômicos e períodos de aceleração ou desaceleração de cada indicador ao longo do tempo.

Exemplo de Interpretação Econômica

Em julho de 2022, observou-se um cenário caracterizado por taxas de juros elevadas, com CDI de 13,15% ao ano e Selic de 13,25% ao ano. No mesmo período, o IPCA registrou variação de -0,68% no mês, caracterizando um episódio de deflação.

Embora os valores não possam ser comparados diretamente devido às diferenças de periodicidade e escala dos indicadores, a combinação desses resultados fornece informações importantes sobre o contexto econômico do período.

A presença simultânea de juros elevados e inflação negativa sugere um ambiente de política monetária restritiva. Em outras palavras, o Banco Central mantinha taxas de juros elevadas como instrumento de controle inflacionário, mesmo após a desaceleração observada nos índices de preços.

Esse contexto pode ser interpretado como um período de elevada pressão monetária, situação em que o custo do crédito tende a permanecer elevado, reduzindo o consumo e os investimentos, enquanto a política econômica busca conter ou estabilizar a inflação.

Para consumidores com operações financeiras indexadas a juros de mercado, períodos caracterizados por CDI e Selic elevados tendem a representar momentos de maior pressão financeira, independentemente do comportamento pontual da inflação em um determinado mês.

2. Qual teria sido o custo acumulado de um financiamento corrigido pelo CDI em comparação com o IPCA?

Durante o desenvolvimento do projeto foi avaliada a possibilidade de simular a evolução de um financiamento indexado ao CDI e compará-lo ao IPCA.

Entretanto, a série escolhida para representar o CDI (SGS 4389) possui como característica a disponibilização da taxa anualizada base 252 dias úteis, sendo particularmente adequada para análises macroeconômicas, comparações temporais entre indicadores e avaliação de volatilidade.

Embora a série atenda adequadamente aos objetivos analíticos deste estudo, a construção de uma taxa mensal efetiva diretamente aplicável à simulação de financiamentos demandaria séries específicas de acumulação financeira ou taxas efetivas de mercado.

Dessa forma, optou-se por não realizar a simulação financeira, preservando a consistência metodológica dos resultados apresentados e evitando interpretações potencialmente inadequadas dos dados.

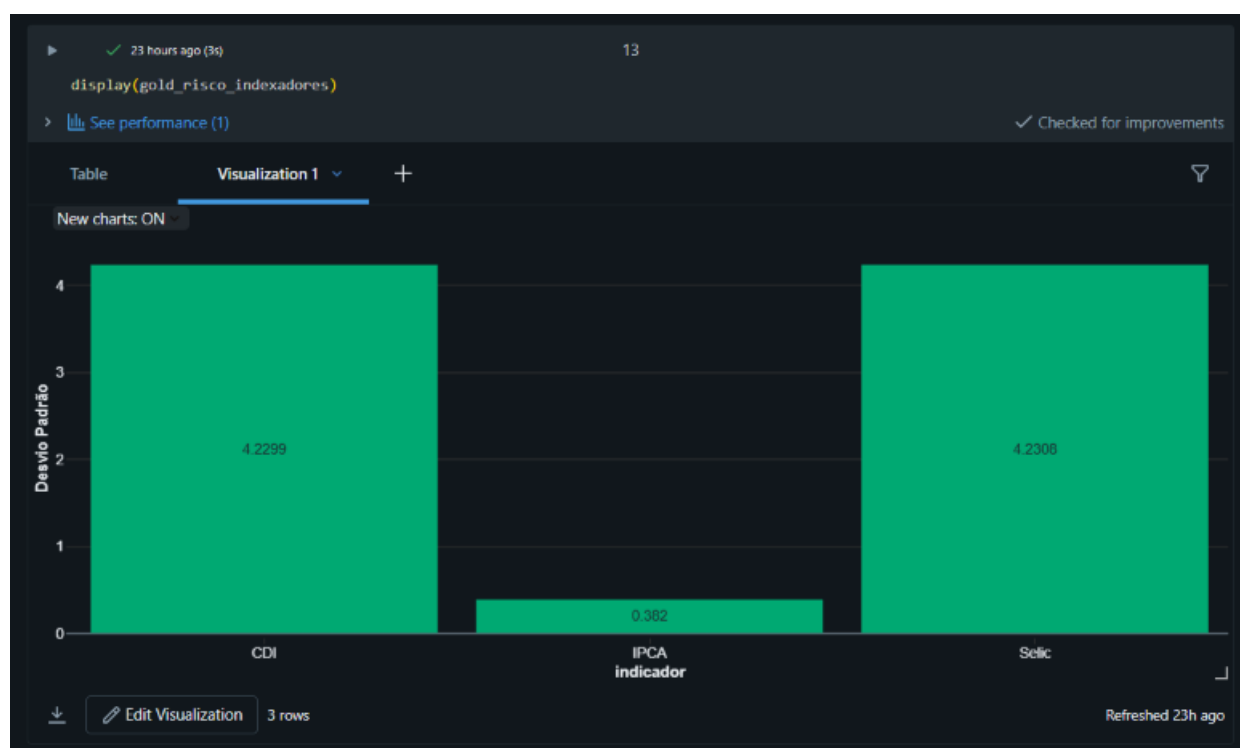
Apesar dessa limitação, os dados utilizados permitiram responder de forma consistente às demais questões de negócio relacionadas à evolução histórica dos indicadores, volatilidade e identificação de períodos de maior pressão monetária.

3. Qual indexador apresentou maior volatilidade no período analisado?

A volatilidade representa o grau de variação de um indicador ao longo do tempo.

Para responder a esta questão, foi utilizada a tabela `gold_risco_indexadores`, construída na camada Gold, contendo métricas estatísticas dos indicadores analisados.

O principal indicador utilizado nesta avaliação é o desvio padrão, amplamente empregado em análises estatísticas e financeiras para mensurar o grau de dispersão dos dados em relação à média.



Interpretação dos Resultados

A volatilidade dos indicadores foi avaliada por meio do desvio padrão, métrica estatística utilizada para mensurar a dispersão dos valores em relação à média.

Os resultados demonstram que CDI e Selic apresentaram níveis de volatilidade significativamente superiores ao IPCA durante o período analisado.

Esse comportamento é esperado, uma vez que CDI e Selic estão diretamente relacionados à política monetária conduzida pelo Banco Central e tendem a sofrer ajustes em momentos de expansão, retração econômica ou mudanças nas estratégias de controle inflacionário.

Por outro lado, o IPCA apresentou menor dispersão ao longo da série histórica, refletindo uma dinâmica mais estável do comportamento dos preços ao consumidor.

Resposta à Pergunta de Negócio

Com base na análise do desvio padrão, conclui-se que CDI e Selic foram os indicadores mais voláteis do período estudado.

O IPCA apresentou comportamento significativamente mais estável, indicando menor variação ao longo dos ciclos econômicos observados entre 2016 e 2026.

4. Em quais períodos um financiamento indexado ao CDI apresentaria maior pressão financeira para o consumidor?

Embora não tenha sido realizada uma simulação financeira direta, é possível identificar períodos de maior pressão financeira a partir do comportamento histórico do CDI.

Como o CDI representa uma referência amplamente utilizada pelo mercado financeiro, períodos caracterizados por taxas mais elevadas tendem a estar associados a maior custo do crédito, encarecimento das operações financeiras e aumento do esforço necessário para manutenção de financiamentos indexados a juros de mercado.

Para responder a essa questão, foi analisada a evolução histórica do CDI ao longo do período estudado.

▶ ✓ 23 hours ago (1s)

17

```
gold_evolucao_indicadores.select(  
  "ano_mes",  
  "cdi_percentual"  
)  
.orderBy(  
  gold_evolucao_indicadores.cdi_percentual.desc()  
)  
.show(10, False)
```

> [📊](#) See performance (1)

```
+-----+-----+  
|ano_mes|cdi_percentual|  
+-----+-----+  
|2026-02|14.9          |  
|2025-11|14.9          |  
|2025-12|14.9          |  
|2026-01|14.9          |  
|2025-08|14.9          |  
|2025-06|14.9          |  
|2025-10|14.9          |  
|2025-07|14.9          |  
|2025-09|14.9          |  
|2025-05|14.65        |  
+-----+-----+  
only showing top 10 rows
```



Observa-se que o indicador atingiu seus menores patamares entre 2020 e 2021, período caracterizado por juros reduzidos e menor custo do crédito.

Interpretação dos Resultados

A análise da evolução histórica do CDI permite identificar os períodos de maior pressão financeira para consumidores expostos a operações indexadas às taxas de juros de mercado.

Observa-se que o indicador atingiu seus menores patamares entre 2020 e 2021, período caracterizado por juros reduzidos e menor custo do crédito.

Esse comportamento esteve associado ao contexto econômico observado durante a pandemia de COVID-19. Diante da desaceleração da atividade econômica, foram adotadas medidas de estímulo monetário, incluindo a redução das taxas básicas de juros, com o objetivo de incentivar o consumo, o investimento e a recuperação econômica.

Como consequência, o CDI atingiu os menores níveis da série histórica analisada, contribuindo para um ambiente de crédito relativamente mais acessível quando comparado aos demais períodos do estudo.

A partir de 2021 ocorre um movimento de forte elevação das taxas de juros, refletindo uma política monetária mais restritiva adotada para controle da inflação.

Os maiores valores da série foram observados entre 2025 e 2026, quando o CDI permaneceu próximo de 14,90% ao ano. Em cenários como esse, financiamentos, empréstimos e demais operações financeiras indexadas aos juros tendem a apresentar custos mais elevados para consumidores e empresas.

8. Conclusão

Este estudo permitiu analisar a evolução histórica dos indicadores CDI, Selic e IPCA entre julho de 2016 e junho de 2026, utilizando uma arquitetura de dados baseada no modelo Medalhão composta pelas camadas Landing, Bronze, Silver e Gold.

A análise demonstrou que CDI e Selic apresentaram comportamentos muito semelhantes ao longo de todo o período estudado, refletindo a forte relação existente entre os indicadores de juros utilizados na política monetária brasileira.

Também foi possível observar movimentos distintos do IPCA, que apresentou oscilações menores e comportamento próprio associado à dinâmica inflacionária da economia brasileira.

Em relação à volatilidade, os resultados indicaram que CDI e Selic foram os indicadores que sofreram maiores variações ao longo do período analisado, enquanto o IPCA apresentou comportamento significativamente mais estável.

A análise dos períodos de maior pressão financeira demonstrou que os maiores níveis do CDI ocorreram entre 2025 e 2026, indicando um ambiente caracterizado por maior custo de crédito e condições mais restritivas para consumidores e empresas dependentes de financiamento.

Por fim, embora a simulação direta de um financiamento indexado ao CDI não tenha sido realizada devido às características metodológicas da série utilizada, o estudo permitiu compreender o comportamento dos principais indicadores econômicos e identificar cenários relevantes para a avaliação do impacto potencial das taxas de juros sobre operações financeiras.

Aplicação dos resultados ao negócio

Sob a perspectiva de uma Cooperativa de Crédito — segmento no qual a autora deste estudo se insere —, os resultados sugerem que o cenário mais recente da série histórica apresenta desafios relevantes para a expansão das carteiras de financiamento imobiliário. Entre janeiro de 2025 e junho de 2026, o CDI mensal evoluiu de 13,15% para 14,15% a.a., tendo atingido 14,90% a.a. em junho de 2025, o que caracteriza um dos patamares mais elevados de toda a série histórica analisada, comparável ao nível observado em 2016.

Esse cenário de juros elevados eleva o custo de captação das Cooperativas de Crédito, cujas fontes de recursos e produtos de crédito costumam estar referenciados, direta ou indiretamente, ao CDI. Como consequência, as taxas ofertadas nas linhas de financiamento imobiliário tendem a se elevar, tornando a contratação menos atrativa para o consumidor final e reduzindo o volume de novas operações — o que torna mais desafiador o crescimento dessa carteira específica.

Nesse contexto, estratégias voltadas exclusivamente para a ampliação da oferta de financiamentos tendem a apresentar resultados limitados. Ganha relevância, portanto, o fortalecimento do relacionamento com os cooperados, a ampliação da base de associados e a diversificação dos produtos financeiros oferecidos pela instituição. A pulverização da carteira — por meio do aumento do número de cooperados associado à oferta de soluções complementares de crédito, investimentos e seguros — contribui para reduzir a dependência de um único produto e para aumentar a resiliência da Cooperativa diante de ciclos econômicos marcados por juros elevados.

Como limitação, cabe destacar que este estudo se restringiu ao período de julho de 2016 a junho de 2026 da série histórica, contemplando um ciclo completo de alta, queda e retomada

da Selic, mas sem comparações com outras instituições do segmento, o que pode ser explorado em pesquisas futuras. Ainda assim, o monitoramento contínuo dos indicadores econômicos analisados pode auxiliar a Cooperativa na definição de estratégias comerciais mais aderentes ao cenário vigente, permitindo equilibrar crescimento, competitividade e gestão de risco.

9. Autoavaliação

Durante o desenvolvimento deste MVP tive a oportunidade de aplicar, na prática, conceitos de Engenharia de Dados, modelagem analítica e análise de dados utilizando uma arquitetura Lakehouse baseada no modelo Medalhão.

Um dos principais aprendizados deste projeto foi desenvolver a capacidade de equilibrar decisões na estruturação de um pipeline de dados completo. Embora houvesse interesse em construir um número maior de tabelas, foi necessário avaliar criteriosamente quais eram efetivamente necessárias em cada etapa — ingestão, tratamento, consolidação e disponibilização — garantindo coerência entre as camadas Bronze, Silver e Gold.

Outro aspecto importante foi o processo de modelagem dos dados. Inicialmente, imaginava que seria necessário construir uma estrutura dimensional mais complexa, porém, ao longo do projeto, compreendi que a modelagem deve ser adequada ao problema que se deseja resolver. Dessa forma, foi adotada uma modelagem dimensional simplificada, composta por uma dimensão temporal (`dim_calendario`) e por uma tabela central de indicadores econômicos (`silver_indicadores_macro`), permitindo organizar os dados para consumo analítico de forma simples, eficiente e compatível com o escopo do MVP.

Considero que um dos pontos fortes do trabalho foi a preocupação em justificar as decisões tomadas ao longo do desenvolvimento. Questões como a consolidação temporal dos indicadores, a diferença de periodicidade entre CDI, Selic e IPCA, a criação da dimensão calendário e a construção das tabelas analíticas da camada Gold foram cuidadosamente avaliadas e documentadas.

Também foi possível compreender a importância da análise crítica dos dados. Durante a construção da camada Gold, identifiquei que a série escolhida para representar o CDI era

adequada para análises macroeconômicas, evolução histórica e avaliação de volatilidade, mas não para a reconstrução direta de uma taxa efetiva de financiamento imobiliário. Em vez de forçar uma conclusão que poderia comprometer a qualidade do estudo, optei por registrar formalmente essa limitação e preservar a consistência metodológica do projeto.

Como oportunidade de evolução futura, o trabalho poderia ser ampliado por meio da utilização de séries específicas de CDI acumulado ou de taxas efetivas de mercado, permitindo a construção de simulações financeiras mais aderentes ao contexto dos financiamentos imobiliários.

Sob a perspectiva de negócio, os resultados obtidos demonstraram como indicadores econômicos como CDI, Selic e IPCA podem apoiar a tomada de decisão em instituições financeiras e cooperativas de crédito. Além disso, foi possível perceber como períodos de juros elevados podem impactar a expansão de carteiras de crédito e influenciar estratégias comerciais relacionadas à concessão de financiamentos.

Por fim, considero que os objetivos propostos foram atingidos. Ao longo do projeto foi possível construir uma arquitetura de dados completa, estruturar informações para consumo analítico, responder às perguntas de negócio inicialmente definidas e transformar dados econômicos em informações capazes de apoiar decisões estratégicas em um contexto real de mercado.

10. Referências

BANCO CENTRAL DO BRASIL. Sistema Gerenciador de Séries Temporais (SGS).

Disponível em:

<https://www3.bcb.gov.br/sgspub/>

Acesso em: 17 set. 2026.

DATABRICKS. Medallion Architecture.

Disponível em:

<https://www.databricks.com/glossary/medallion-architecture>

Acesso em: 17 set. 2026.

GITHUB. Repositório do Projeto MVP Financiamento CDI x IPCA.

Disponível em:

<https://github.com/miriamstal26-rgb/mvp-financiamento-cdi-ipca>

Acesso em: 17 set. 2026.